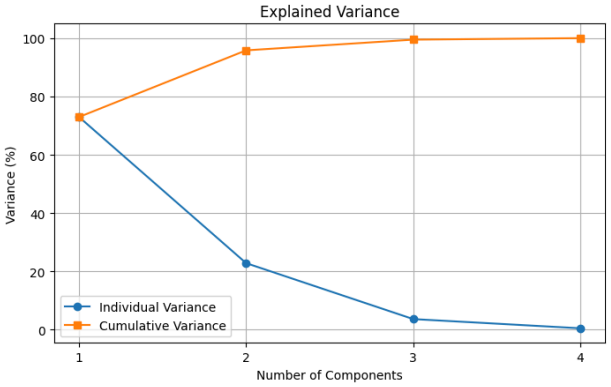
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Standardize the data
X = StandardScaler().fit_transform(X)

# Apply PCA
pca = PCA()
X_pca = pca.fit_transform(X)

# Calculate variance
ratios = pca.explained_variance_ratio_
cumulative = np.cumsum(ratios)

# Create result table
table = pd.DataFrame({
    "Components": [1, 2, 3, 4],
    "Output Dimension": [1, 2, 3, 4],
    "Explained Variance (%)": ratios * 100,
    "Cumulative Variance (%)": cumulative * 100
})

print("PCA COMPONENT ANALYSIS")
print("-" * 75)
print(table.to_string(index=False, formatters={
    "Explained Variance (%)": "{:.2f}".format,
    "Cumulative Variance (%)": "{:.2f}".format
}))

# Create plots
fig = plt.figure(figsize=(14, 9))

# 1. Explained Variance
ax1 = fig.add_subplot(2, 2, 1)

ax1.plot(
    [1, 2, 3, 4],
    ratios * 100,
    marker="o",
    label="Individual Variance"
)
```

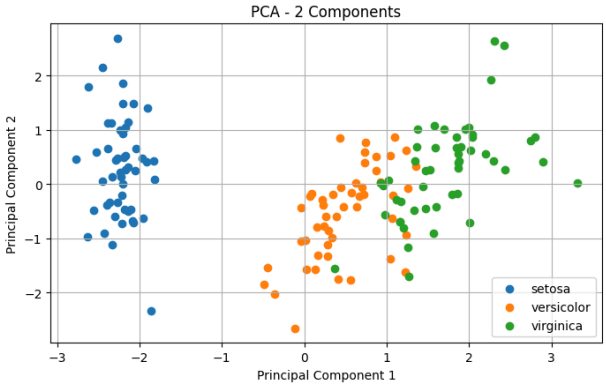
```
)

ax1.plot(
    [1, 2, 3, 4],
    cumulative * 100,
    marker="s",
    label="Cumulative Variance"
)

ax1.set_title("Explained Variance")
ax1.set_xlabel("Number of Components")
ax1.set_ylabel("Variance (%)")
ax1.set_xticks([1, 2, 3, 4])
ax1.grid()
ax1.legend()

# 2. PCA with 2 Components
ax2 = fig.add_subplot(2, 2, 2)

for i in range(3):
    ax2.scatter(
        X_pca[y == i, 0],
        X_pca[y == i, 1],
        label=iris.target_names[i]
    )

ax2.set_title("PCA - 2 Components")
ax2.set_xlabel("Principal Component 1")
ax2.set_ylabel("Principal Component 2")
ax2.legend()
ax2.grid()

# 3. PCA with 3 Components
ax3 = fig.add_subplot(2, 2, 3, projection="3d")

for i in range(3):
    ax3.scatter(
        X_pca[y == i, 0],
        X_pca[y == i, 1],
        X_pca[y == i, 2],
        label=iris.target_names[i]
    )

ax3.set_title("PCA - 3 Components")
ax3.set_xlabel("PC1")
ax3.set_ylabel("PC2")
ax3.set_zlabel("PC3")
ax3.legend()

# 4. Information Preservation
ax4 = fig.add_subplot(2, 2, 4)
```

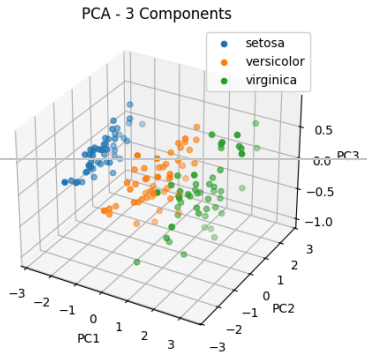
```
ax4.bar(
    [1, 2, 3, 4],
    cumulative * 100
)

ax4.set_title("Information Preservation")
ax4.set_xlabel("Number of Components")
ax4.set_ylabel("Cumulative Variance (%)")
ax4.set_xticks([1, 2, 3, 4])
ax4.grid(axis="y")

plt.tight_layout()
plt.show()

# Analysis
print("\nANALYSIS")
print("-" * 75)

for n in [1, 2, 3, 4]:
    print(
        f"{n} component(s): "
        f"{cumulative[n-1] * 100:.2f}% information preserved"
    )

# Conclusion
print("\nCONCLUSION")
print("-" * 75)
print("Using 2 components reduces the dimension from 4 to 2")
print(f"while preserving {cumulative[1] * 100:.2f}% of the information.")
print("Therefore, 2 components provide a good balance between")
print("dimensionality reduction and information preservation.")
```

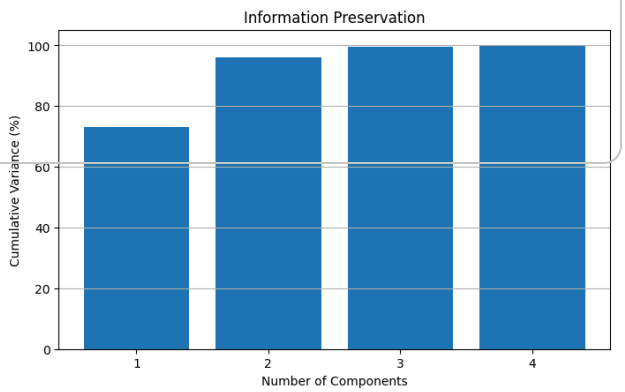
```
PCA COMPONENT ANALYSIS
--------------------------------------------------------------------------------
```

Components	Output Dimension	Explained Variance (%)	Cumulative Variance (%)
1	1	72.96	72.96
2	2	22.85	95.81
3	3	3.67	99.48
4	4	0.52	100.00










```
ANALYSIS
--------------------------------------------------------------------------------
1 component(s): 72.96% information preserved
2 component(s): 95.81% information preserved
3 component(s): 99.48% information preserved
4 component(s): 100.00% information preserved

CONCLUSION
--------------------------------------------------------------------------------
Using 2 components reduces the dimension from 4 to 2
while preserving 95.81% of the information.
Therefore, 2 components provide a good balance between
dimensionality reduction and information preservation.
```